

Documentación de Scripts - Task 1 (Clasificación de Misoginia en Canciones)

Estructura de archivos

Script	Descripción
<code>analyze_token_lengths.py</code>	Análisis de longitudes de tokens y sesgo por etiqueta
<code>bert_model_configs.py</code>	Configuraciones de modelos BERT
<code>bert_based_comparison.py</code>	Entrenamiento comparativo de modelos BERT
<code>llm_model_configs.py</code>	Configuraciones de LLMs con QLoRA
<code>llm_qlora_comparison.py</code>	Entrenamiento comparativo de LLMs con QLoRA
<code>train_kfold.py</code>	Entrenamiento con validación cruzada K-Fold
<code>ensemble_kfolds.py</code>	Ensemble de modelos K-Fold para inferencia
<code>ensemble_test.py</code>	Ensemble ponderado de múltiples modelos
<code>evaluate_on_test.py</code>	Evaluación de modelos en conjunto de test

1. `analyze_token_lengths.py`

Propósito: Analiza la distribución de longitudes de tokens y detecta sesgos entre longitud de texto y etiquetas de clase.

Funciones:

Función	Descripción
<code>sanitize_tokenizer_name(name: str)</code>	Convierte el ID del tokenizer en un sufijo válido para el sistema de archivos
<code>summarize_lengths(lengths: np.ndarray)</code>	Calcula estadísticas descriptivas (media, std, percentiles) de las longitudes
<code>coverage_at_thresholds(lengths, thresholds)</code>	Calcula el porcentaje de muestras cubiertas por cada umbral de <code>max_len</code>
<code>build_length_buckets(lengths, thresholds)</code>	Construye buckets disjuntos de longitud para análisis
<code>infer_label_type(series)</code>	Infiere el tipo de etiqueta (binaria numérica, texto, multiclase)
<code>compute_positive_mask(labels)</code>	Crea máscara booleana para la clase positiva en clasificación binaria
<code>print_summary(summary, coverages)</code>	Imprime resumen estadístico y coberturas

Función	Descripción
<code>print_label_bias_report(lengths, labels, thresholds, label_col)</code>	Genera reporte de sesgo etiqueta-longitud
<code>main()</code>	Función principal con argumentos CLI

Uso:

```
python analyze_token_lengths.py --csv-path data/task1/train.csv --tokenizer dccuchile/bert-base-spanish-wwm-uncased
```

2. bert_model_configs.py

Propósito: Define las configuraciones de arquitectura y entrenamiento para modelos basados en BERT.

Clases:

Clase	Descripción
<code>ModelConfig</code>	Dataclass con parámetros de arquitectura (pooling, dropout), tokenización (<code>max_len</code>), entrenamiento (lr, epochs, scheduler) y Focal Loss

Configuraciones predefinidas (MODEL_CONFIGS):

Modelo	ID	max_len	learning_rate
DistilBETO	dccuchile/distilbert-base-spanish-uncased	512	2e-5
BETO	dccuchile/bert-base-spanish-wwm-cased	512	5e-6
MarIA	IsGarrido/roberta-base-bne	512	3e-5
XML-R	xlm-roberta-base	512	2e-5
Robertuito	pysentimiento/robertuito-base-uncased	128	2e-5
LongFormer	mrm8488/longformer-base-4096-spanish	1024	2e-5

3. bert_based_comparison.py

Propósito: Entrena y compara múltiples modelos BERT para clasificación binaria de misoginia. Genera tabla comparativa para paper.

Funciones:

Función	Descripción
<code>main(augment=False)</code>	Función principal que ejecuta la comparativa
<code>compute_metrics(pred)</code>	Calcula accuracy, F1-macro, precision y recall
<code>find_best_threshold(true_labels, probs, step)</code>	Busca el umbral óptimo para maximizar F1-macro
<code>make_tokenize_fn(tokenizer, cfg)</code>	Factory de tokenización con truncamiento inteligente head+tail (75%/25%)
<code>load_model_with_config(model_name, cfg, device)</code>	Carga modelo con configuración de pooling personalizada
<code>make_training_args(cfg, checkpoints_path)</code>	Construye <code>TrainingArguments</code> desde la configuración

Características: - Truncamiento inteligente: 75% cabeza + 25% cola del texto
- Pesos de clase para desbalance - Focal Loss configurable - Early stopping - Data augmentation opcional

4. `llm_model_configs.py`

Propósito: Define configuraciones para LLMs grandes con fine-tuning QLoRA.

Clases:

Clase	Descripción
<code>LLMConfig</code>	Dataclass con parámetros LoRA (r, alpha, target_modules), cuantización y entrenamiento

Configuraciones predefinidas (LLM_CONFIGS):

Modelo	ID	max_len	lora_r	batch_size
Qwen-2.5-7B	Qwen/Qwen2.5-7B	1024	8	1
Llama-3-8B	meta-llama/Meta-Llama-3-8B	1024	8	1
Llama-3.2-1B	meta-llama/Llama-3.2-1B	1024	8	16
Qwen-2.5-0.5B	Qwen/Qwen2.5-0.5B	1024	8	16
Qwen-2.5-3B	Qwen/Qwen2.5-3B	1024	8	4
Llama-3.2-3B	meta-llama/Llama-3.2-3B	1024	8	8

5. `llm_qlora_comparison.py`

Propósito: Entrena y compara LLMs con QLoRA (4-bit quantization + LoRA) para clasificación de misoginia.

Funciones:

Función	Descripción
<code>compute_metrics(pred)</code>	Calcula métricas de evaluación
<code>make_tokenize_fn(tokenizer, max_len)</code>	Factory de función de tokenización

Características: - Cuantización 4-bit con BitsAndBytes (NF4) - LoRA para fine-tuning eficiente - Compatible con GPUs de 12GB VRAM - Focal Loss para desbalance de clases

6. `train_kfold.py`

Propósito: Entrenamiento con validación cruzada K-Fold estratificada, evitando data leakage.

Funciones:

Función	Descripción
<code>make_tokenize_fn(tokenizer, cfg)</code>	Factory de tokenización con truncamiento head+tail
<code>run_kfold()</code>	Ejecuta el entrenamiento K-Fold completo

Características: - **Anti-leakage:** El split se hace por ID de canción (no por fila), asegurando que versiones aumentadas de una canción no aparezcan en train y val simultáneamente - Validación solo con muestras originales - Pesos de clase calculados por fold - Guarda modelo y tokenizer por cada fold

7. `ensemble_kfolds.py`

Propósito: Realiza inferencia en test usando ensemble de modelos K-Fold con sliding window chunking.

Funciones:

Función	Descripción
<code>chunk_tokens(token_ids, max_len, stride)</code>	Divide tokens en chunks superpuestos con CLS/SEP
<code>get_chunked_probability(text, tokenizer, model, ...)</code>	Obtiene probabilidad de clase M con max-pooling sobre chunks
<code>get_fold_probabilities(fold_idx, df_test, cfg)</code>	Obtiene probabilidades de un fold específico
<code>main()</code>	Promedia probabilidades de todos los folds y genera predicciones

Características: - Sliding window con stride configurable (256 por defecto) - Max-pooling sobre chunks para textos largos - Promedio de probabilidades entre folds

8. ensemble_test.py

Propósito: Crea un ensemble ponderado de múltiples modelos diferentes para predicción final.

Funciones:

Función	Descripción
<code>chunk_tokens(token_ids, max_len, stride)</code>	Divide tokens en chunks superpuestos
<code>get_chunked_probability(...)</code>	Probabilidad con sliding window y max-pooling
<code>get_model_probabilities(name, df_test)</code>	Obtiene probabilidades de un modelo específico
<code>main()</code>	Ejecuta ensemble ponderado con power averaging

Características: - **Power Averaging:** Eleva probabilidades a una potencia (1.5) para premiar confianza - Pesos configurables por modelo (LongFormer: 0.34, MarIA: 0.33, DistilBETO: 0.33) - Evaluación automática si existen etiquetas de test

9. evaluate_on_test.py

Propósito: Evalúa todos los modelos entrenados en el conjunto de test y genera tabla comparativa.

Funciones:

Función	Descripción
<code>chunk_tokens(token_ids, max_len, stride)</code>	Chunking de tokens
<code>get_chunked_probability(...)</code>	Inferencia con sliding window
<code>run_inference(model_path, texts, true_labels, ...)</code>	Ejecuta inferencia completa para un modelo

Características: - Usa thresholds óptimos encontrados en validación - Guarda predicciones individuales por modelo - Genera reporte de clasificación y matriz de confusión - Sliding window con stride adaptativo (256 para BERT, 64 para Robertuito)

Flujo de trabajo típico

1. `analyze_token_lengths.py` → Análisis exploratorio de datos
2. `bert_model_configs.py` → Definir configuraciones
3. `bert_based_comparison.py` → Entrenar y comparar modelos (validación)
4. `train_kfold.py` → Entrenamiento robusto con K-Fold
5. `evaluate_on_test.py` → Evaluar modelos individuales en test
6. `ensemble_kfolds.py` → Ensemble de folds de un modelo
7. `ensemble_test.py` → Ensemble final de múltiples modelos